Grace Jungclas, MA, AT Ret
CS 300
August 9, 2026

CS 300 Project 1

Runtime Analysis

The following tables are analyses of the runtime of code that reads a data file and creates course objects, then inserts them into the data structure, where c is a constant representing the average/max number of prerequisites per course

Vector Pseudocode- Worst Case Runtime	Line Cost	# Times Executes	Total Cost
While there are lines to read	1	n+1	n+1
Read line and split into tokens	1	n	n
If token count < 2	1	n	n
OUTPUT error	1	0	0
ELSE {	1	n	n
instantiate object	1	n	n
Set courseNumber = token[0]	1	n	n
Set courseName = token[1]	1	n	n
For each token from index 2	1	$c \cdot n$	$c \cdot n$
Add token to preqs	1	$c \cdot n$	$c \cdot n$
PUSH currentCourse into vector	1	n	n
		Total Cost	(7+2c)n + 1
		Runtime	O(n)

Hash Table Pseudocode – Worst Case Runtime	Line Cost	# Times Executes	Total Cost
While there are lines to read	1	n+1	n+1
Read line and split into tokens	1	n	n
If token count < 2	1	n	n
OUTPUT error	1	0	0
ELSE {	1	n	n
instantiate object	1	n	n
Set courseNumber = token[0]	1	n	n
Set courseName = token[1]	1	n	n
For each token from index 2	1	$c \cdot n$	$c \cdot n$
Add token to preqs	1	$c \cdot n$	$c \cdot n$
CALL courseTable.Insert(currentCourse)	n	n	n^2
		Total Cost	n^2 +(6+2c)n + 1
		Runtime	$O(n^2)$

Binary Search Tree Pseudocode – Worst Case Runtime	Line Cost	# Times Executes	Total Cost
While there are lines to read	1	n+1	n+1
Read line and split into tokens	1	n	n
If token count < 2	1	n	n
OUTPUT error	1	0	0
ELSE {	1	n	n
instantiate object	1	n	n
Set courseNumber = token[0]	1	n	n
Set courseName = token[1]	1	n	n
For each token from index 2	1	$c \cdot n$	$c \cdot n$
Add token to preqs	1	$c \cdot n$	$c \cdot n$
CALL courseTable.Insert(currentCourse)	n	n	n^2
		Total Cost	$n^2 + (6+2c)n + 1$
		Runtime	$O(n^2)$

Evaluation

To determine the most effective data structure for the Academic Advising Assistance Program at ABCU, three primary options—Vectors, Hash Tables, and Binary Search Trees (BST)—were evaluated against the project's core requirements: loading file data, performing individual course lookups with prerequisites, and outputting an alphanumerically sorted course schedule.

The Vector (dynamic array) provides a guaranteed linear ingestion, as appending course objects to the end of a contiguous vector operates in amortized $O(1)$ time per item (Cormen et al., 2022). Reading and loading n courses takes a predictable $O(n)$ worst-case runtime regardless of input ordering, avoiding issues like hash collisions or tree traversals during file ingestion. Additionally, vectors store elements sequentially in continuous memory blocks, offering high CPU cache locality and lower code complexity by avoiding dynamic node allocation, pointer management, and recursive structures (Weiss, 2014). However, searching for a specific course or verifying prerequisites requires iterating sequentially through the elements until a match is found, resulting in an $O(n)$ search runtime per query (Sedgewick & Wayne, 2011). Furthermore,

because vectors maintain insertion order rather than key order, an external sorting algorithm must be executed on the vector to print an alphanumerically ordered schedule, adding $O(n \log n)$ time penalty prior to output (Cormen et al., 2022).

The Hash Table excels at constant time searches, utilizing a uniform hash function to locate individual courses in average-case $O(1)$ time (Weiss, 2014). This provides near-instantaneous lookup when requesting individual course details and prerequisites. On the other hand, separate chaining introduces significant disadvantages under worst-case conditions. If severe hash collisions occur, bucket chains degrade into linear list traversals, causing the total file-loading runtime for n items to slip to $O(n^2)$ (Cormen et al., 2022). Hash tables also store nodes according to key hashes rather than natural alphanumeric ordering (Sedgewick & Wayne, 2011). Generating an ordered schedule requires extracting all nodes into an auxiliary structure and sorting them, incurring $O(n \log n)$ time and added memory overhead. Finally, managing node pointers (*next) and bucket arrays increases overall source code complexity.

The Binary Search Tree (BST) naturally structures data based on course keys during insertion. Performing a standard in-order traversal prints the entire course schedule alphanumerically in linear $O(n)$ time without requiring external sorting algorithms or additional memory allocations. Searching for individual courses or verifying prerequisites operates in logarithmic time ($O(n \log n)$) on average by traversing tree branches, striking an effective balance between lookup speed and structural ordering (Weiss, 2014). The primary drawback of a standard BST lies in its susceptibility to input data ordering during ingestion. If course records are loaded in pre-sorted order, an unbalanced tree degenerates into a linear chain of height $h = n$, resulting in an $O(n^2)$ worst-case loading time (Cormen et al., 2022). Implementing a BST also

requires managing dynamic memory pointers (*left, *right) and writing recursive logic for insertion, search, and traversal operations.

Data Structure Recommendation & Justification

The Binary Search Tree (BST) is recommended as the primary data structure for the ABCU Academic Advising Assistance Program. This recommendation is grounded in the BST's direct alignment with the advising team's core operational needs: looking up individual course details/prerequisites and outputting an alphanumerically sorted schedule of all courses. While a Hash Table offers faster-than-average search times ($O(1)$ vs. $O(\log n)$), it stores data out of order, requiring extra extraction and sorting steps to produce a schedule. A BST natively satisfies both requirements through logarithmic search operations ($O(n \log n)$) and linear-in-order traversal printing($O(n)$) (Main, 2012).

From an operational standpoint, choosing a BST ensures that course lookups scale efficiently as ABCU's curriculum grows, avoiding the linear lookup delays associated with flat vectors (Weiss, 2014). The primary structural risk of a standard BST—worst-case $O(n^2)$ build time caused by pre-sorted input—can be easily managed by shuffling input records prior to insertion or adapting the tree logic to self-balancing variants in future program updates (Cormen et al., 2022). Overall, the Binary Search Tree offers the most balanced, elegant, and maintainable framework to meet every requirement specified by the academic advisors.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. Retrieved from https://www.yumpu.com/en/document/read/67420438/introduction-to-algorithms-4th-edition-leiserson-stein-rivest/7

Main, M. (2012). *Data Structures and Other Objects Using C++* (4th ed.). Pearson. Retrieved from https://www.pearson.com/en-us/subject-catalog/p/data-structures-and-other-objects-using-c/P200000003303/9780132129480?srsltid=AfmBOopx8zhNQU6CbDAlTesE4W45jNqeCpMYedgARkagVUvEdf_WDM_Z

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional. Retrieved from https://algs4.cs.princeton.edu/home/

Weiss, M. A. (2014). *Data Structures and Algorithm Analysis in C++* (4th ed.). Pearson. Retrieved from chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.uoitc.edu.iq/images/documents/informatics-institute/Competitive_exam/DataStructures.pdf